

PART 2 OF 5. DEFINE WHICH ITEMS BELONG IN THE REPORT.

Turn the question into a rule

"Is this on track?" is a useful question to ask a person. A script needs a more specific instruction, such as "show open items whose due date has passed." You can automate that comparison and still ask the owner what happened.

In this part, you'll write two example rules, decide what to do with missing information, and compare the results with records you can read yourself. The time windows below are choices for this exercise, not a claim about the right reminder schedule for every team.

1 Write down the date and state definitions

Choose one project and the kinds of items to include. List the exact state values that count as active and complete. If your tracker uses "Resolved" differently from "Closed," decide how each should behave before writing the rule.

For this exercise, treat due dates as calendar dates in America/Los_Angeles. An item due today isn't overdue. It becomes overdue the next calendar day. If your source uses timestamps, agree on the conversion or deadline convention instead of dropping the time without checking.

Copy and complete this rule sheet for your own tracker:

- ☐ Project and item types included:
- ☐ Stable item ID and link fields:
- ☐ Active states:
- ☐ Completed states:
- ☐ Due-date field and time zone:
- ☐ Owner account field and verified recipient mapping:
- ☐ State-history field or source:
- ☐ Report date and time:

Use a fixed report date when testing. Otherwise a test that passes today could return a different result tomorrow.

EXAMPLE

Fictional rule sheet entry: active states are "New" and "In progress." Complete states are "Resolved" and "Closed." The due-date field is called "Target Date" and stores a calendar date. The report date is fixed at September 22, 2026, 9:00 AM in America/Los_Angeles for every test run.

WHY THIS MATTERS: every run repeats these definitions exactly as you wrote them. If "Resolved" means finished to half the team and not to the other half, the report is wrong the same way every day until somebody notices.

2 Keep the two reasons for a reminder separate

Overdue: include an item when its state is active and its due date is earlier than the report date.

Due soon, unchanged: include an active item due from the report date through seven calendar days later, inclusive, when reliable state history shows no state change for at least fourteen days.

EXAMPLE

With a report time of September 22, 2026 at 9:00 AM in Los Angeles, an active item due September 21 is overdue. An active item due September 29 qualifies for the second list if its last state change was September 8 at 9:00 AM or earlier. September 30 is outside that window.

The second rule requires state history. The current state and due date alone can't show how long the state has been unchanged. Use a documented last-state-change timestamp or complete transition history.

If you only have saved daily snapshots, you can report that the state was the same in those snapshots. You can't rule out a change and reversal between them. Keep that observation separate from this fourteen-day rule.

Don't call the second list "stuck." Work can progress without a state change. The proposed message should ask for an update without asserting that nothing happened.

WHY THIS MATTERS: an item due next week and an item three weeks late need different messages. Collapse them into one alert and every reminder reads like a complaint, so people start filtering the whole thing.

3 Decide what to do with records you can't evaluate

Start by checking the stable item ID. A record without one goes into a review list. A known completed item is excluded. For the remaining items, check the state, date, and owner before deciding whether to prepare a message.

Use these outcomes for the exercise:

- ☐ Missing date, missing owner, unknown state, or unverified recipient: hold for review and show the reason. Don't propose a message.
- ☐ Missing state history: an overdue item can still qualify. A due-soon item goes to review because you can't evaluate the fourteen-day condition.
- ☐ Exact duplicate record: evaluate that item once and report the duplicate.
- ☐ Same item ID with conflicting values: hold that item for review. Don't choose whichever copy arrived first.
- ☐ Outside the rules: don't propose a message, but retain the exclusion reason so you can check it.

EXAMPLE

A known completed item with an old due date is excluded. If the tracker says "In progress" but the owner tells you the work shipped, hold the proposed message and confirm which record needs correcting. Don't change the report to say "Complete" without updating or resolving the source.

WHY THIS MATTERS: a script that silently drops the records it can't read is worse than no script, because you start trusting the list. Anything it can't evaluate should appear in the review list with the reason attached.

4 Retrieve the records without hiding the gaps

For the first preview, retrieve the agreed set of items with the fields on your rule sheet. Include records with missing owners or dates so they appear in the review list. A filter that removes them before you start would hide those gaps.

Ask your system owner or code assistant to map the request to the approved export or API. Jira, Azure DevOps, and other trackers have different field names, query syntax, permissions, and ways to retrieve history. A query from another team's project isn't ready to use until those choices match yours.

EXAMPLE

Prompt to adapt: "Using the approved documentation and field mapping, read the items in this project scope. Include IDs, links, titles, owner IDs, states, due dates, and the state-change data needed by my rules. Retrieve all result pages. Report an incomplete read as a failure, not as a complete list. Make no writes and send no messages."

This is a specification for code, not a command you can run as written. Start with a saved fictional dataset if live access isn't ready. Have the person reviewing the implementation check the field mapping, result paging, and error handling before connecting it to company records.

WHY THIS MATTERS: the query is the cheap part, which is why it comes fourth. A filter that removes items with no owner makes the output look clean and hides the exact records you need to fix.

5 Check both included and excluded items

Read the output next to the source. For each proposed message, check the item ID, recipient, state, date, and reason. Then inspect some items that didn't make the list. A report can contain correct rows and still miss work you expected it to include.

Use these fictional cases to review the logic before Part 3:

- ☐ Active and due yesterday: overdue.
- ☐ Active and due today, with fourteen days of unchanged-state history: due soon, unchanged.
- ☐ Active and due tomorrow, with a state change yesterday: not selected.
- ☐ Complete and due last month: excluded.
- ☐ Active with no due date: review, no proposed message.
- ☐ Active and due tomorrow with no state history: review, no proposed message.

Record the number of source rows, unique items, proposed messages, review items, excluded items, and duplicate rows. Every unique item should have one outcome. If a read fails halfway through, label the run incomplete and fix that failure before using its totals.

EXAMPLE

You expected eleven overdue items and the report shows nine. The two missing items are in a state called "Blocked," which never made it onto your active list. The rule did what you wrote. The rule sheet was incomplete.

WHY THIS MATTERS: reading only the items that made the list tells you nothing about the ones that should have. Checking both directions is how you catch a rule that's quietly excluding half the project.

Keep your completed rule sheet beside the output. You should be able to point to an item and explain why it was selected, excluded, or held for review.

Part 3 uses a fictional test set and a daily review sheet to check the script before anyone receives a message.